



Técnicas Digitales II

Compilado, Ensamblado, Enlazado y Carga

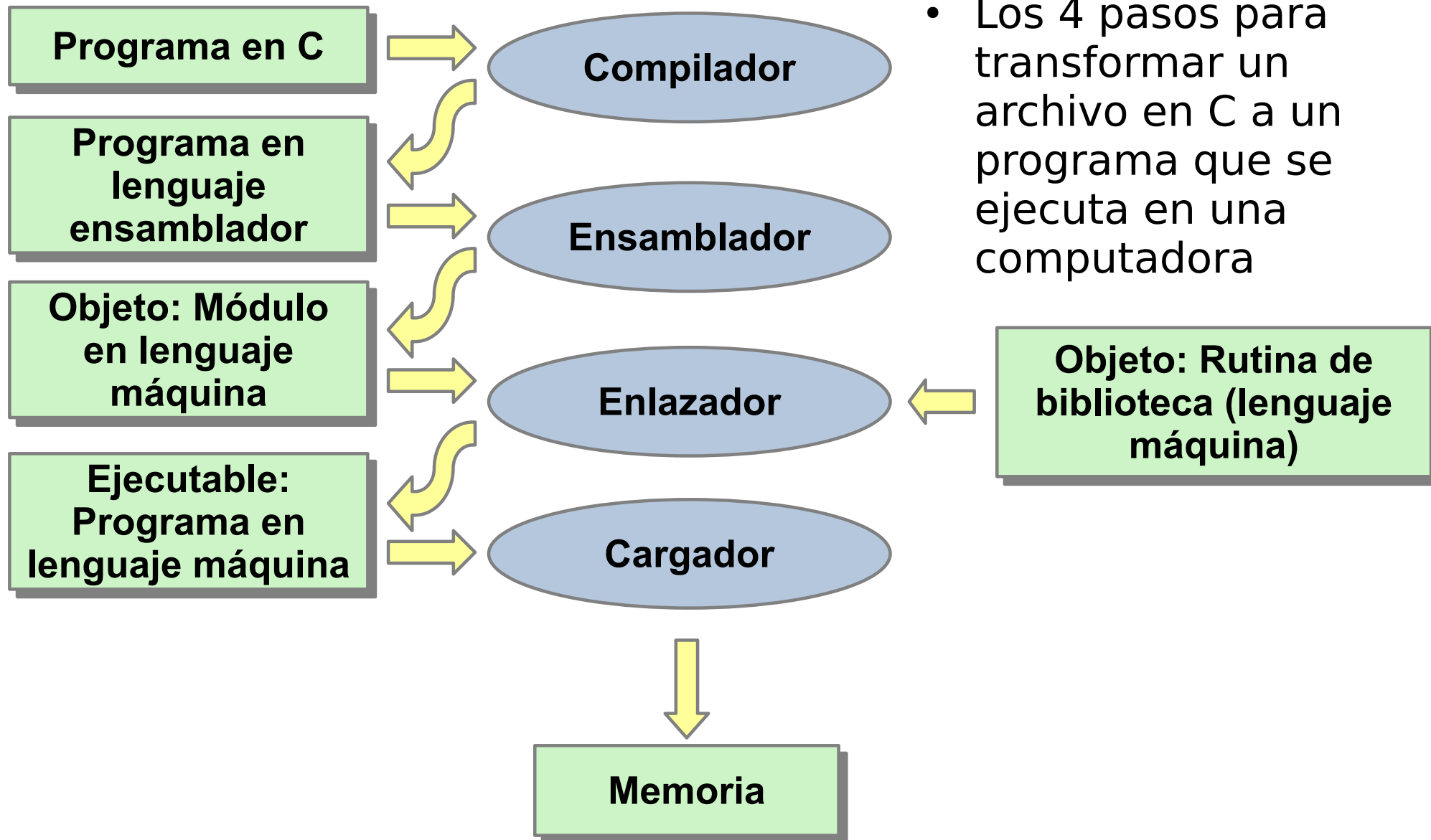
- Bibliografía

Computer Organization and Design ARM Edition, 2017
Capítulo 2.12

- Soporte

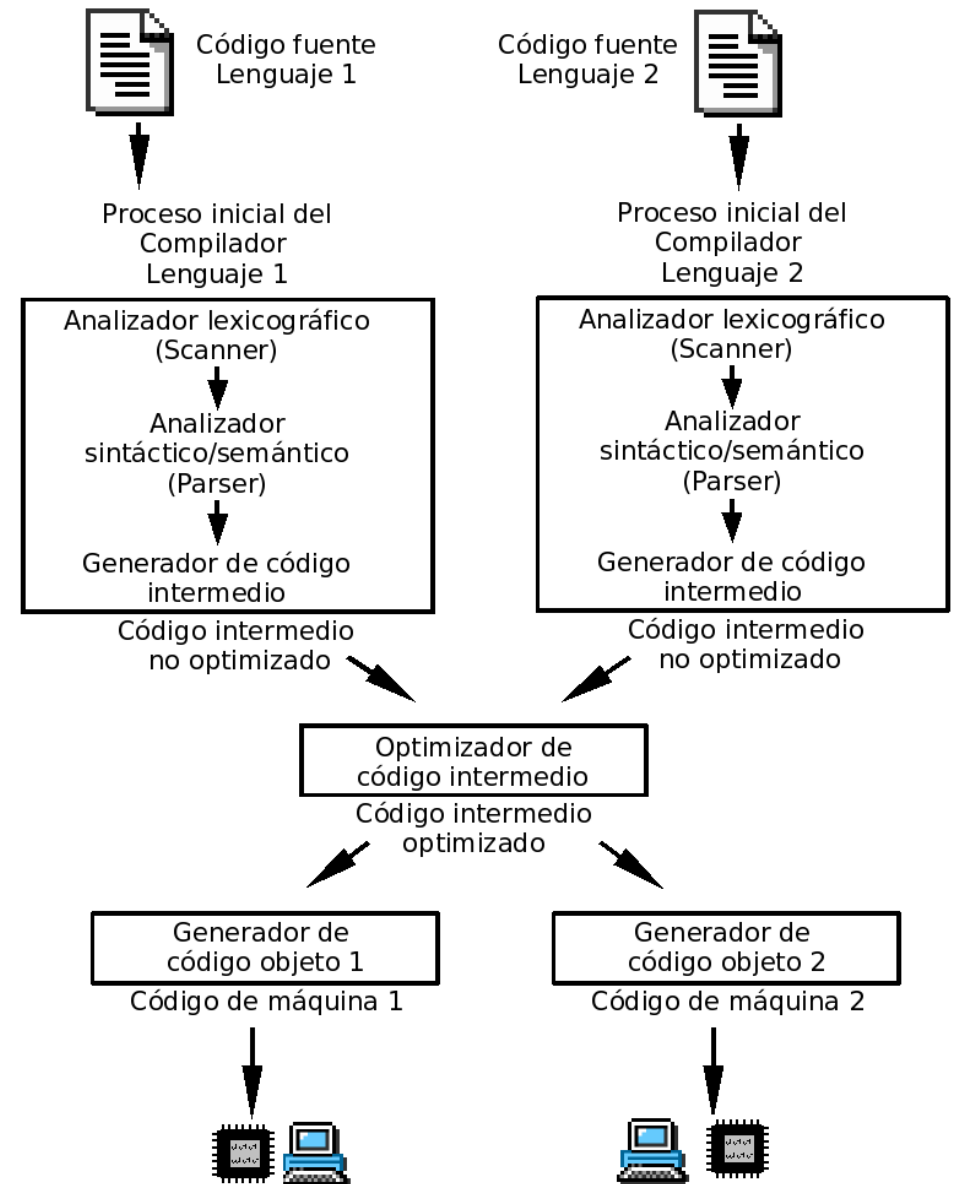
Harris & Harris. Digital design and computer architecture:
ARM edition. Elsevier, 2015. Capítulo 6.

Traducción e inicio de un programa



Compilador

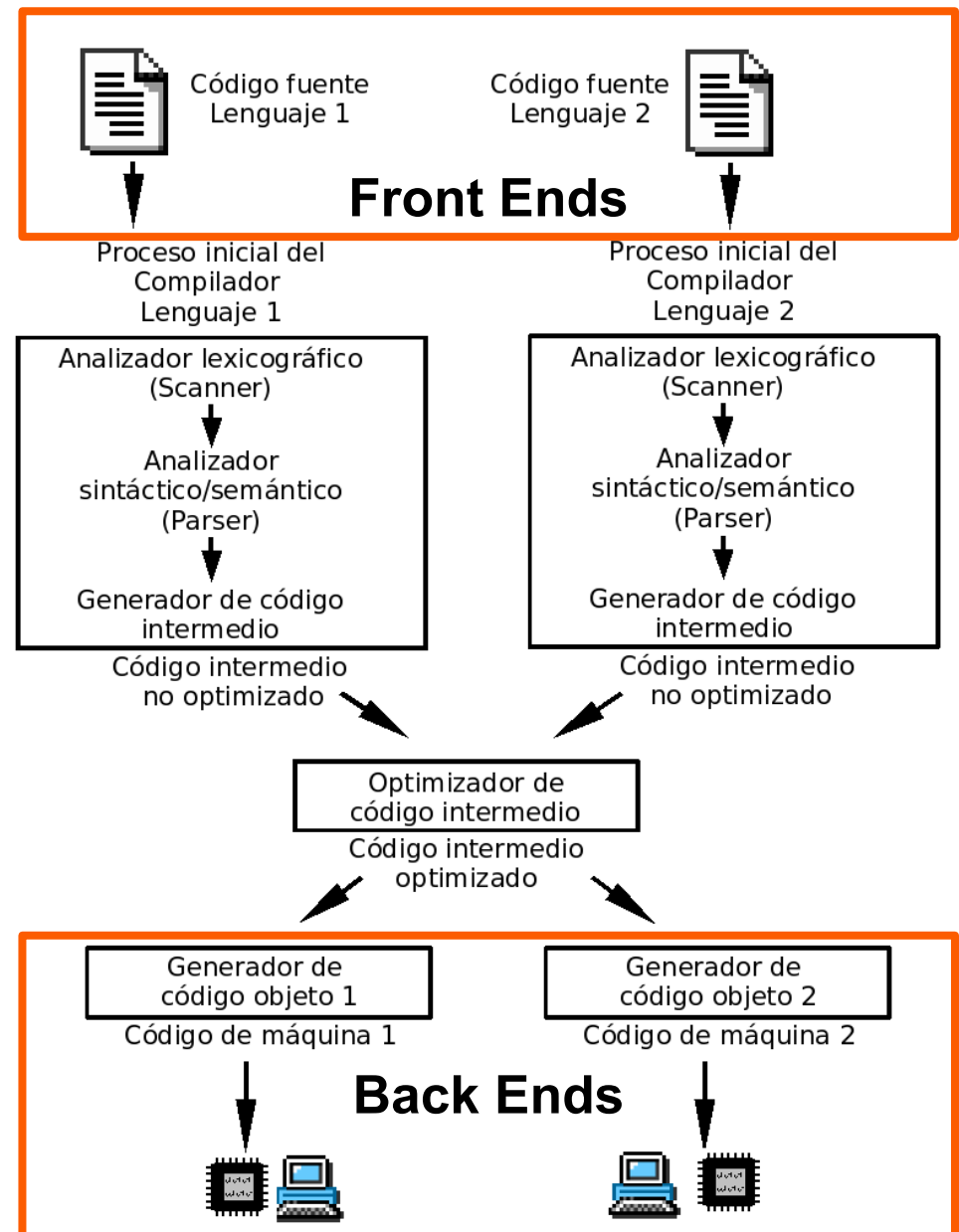
- El compilador es básicamente un traductor, traduce un lenguaje a otro lenguaje.
- En general el compilador traduce un código escrito en lenguaje de alto nivel a Ensamblador, una forma simbólica de lo que la máquina entienda .



Compilador

Un Compilador se puede simplificar dividiéndolo en dos partes

- **Front-Ends** Es el encargado de parsear el código fuente y traducirlo a un lenguaje intermedio.
- **Back-Ends** se traduce el lenguaje intermedio a ensamblador (o código objeto).
- Un compilador puede tener varios Front-Ends y varios Back-Ends.



Compilador

El proyecto GNU utiliza el GCC, compilador ampliamente difundido y libre.

- Este compilador, posee un nivel intermedio o Middle-Ends, esta etapa realiza una serie de optimizaciones independiente del lenguaje original e independiente de la arquitectura, utilizando un lenguaje intermedio (RTL).
- Finalmente este código se transfiere al Back-Ends



GCC compilador escrito para el proyecto GNU.

- **Front Ends** → Oficiales: C, C++, Objective-C, Fortran, Ada, Go y D junto con otros en desarrollo como Pascal, Cobol, etc.
- **Back Ends** → Múltiples arquitectura como ARM, IA-64, MIPS, Motorola M680x0, PowerPC, x86, x86-64, RISC-V, etc

Compilador

```
int f,g,y;

int sum(int a,int b) {
    return (a+b);
}

int main(void)
{
    f=2;
    g=3;
    y=sum(f,g);
    return y;
}
```

```
gcc -01 -S prog.c -o prog.s
```

```
.text
.global sum
.type sum, %function
sum: add r0, r0, r1
     bx lr
.global main
.type main, %function
main: push {r3, lr}
      mov r0, #2
      ldr r3, .L3
      str r0, [r3, #0]
      mov r1, #3
      ldr r3, .L3+4
      str r1, [r3, #0]
      bl sum
      ldr r3, .L3+8
      str r0, [r3, #0]
      pop {r3, pc}
.L3:  .word f
      .word g
      .word y
```



Ensamblado

- Un ensamblador convierte el código ensamblador en un archivo objeto que contiene el código en lenguaje máquina.
- Dentro de sus características:
 - Transforma mnemónicos (como ADD) en sus correspondientes códigos de operación binarios.
 - La utilización de Pseudofunciones, funciones de ensamblador que no existen como tal en código máquina, sino que son una variación de otra instrucción.
 - Además de permitir usar mnemónicos, se pueden usar distintas bases numéricas o etiquetas en lugar de direcciones de memoria



Ensamblado

- En el Ensamblado se realizan dos pasadas al código
 - En la primera, se asignan las direcciones de la instrucciones y se encuentran los símbolos (etiquetas y nombre de variables globales), se crea entonces una tabla de estos símbolos con los nombres y sus direcciones.
 - En la Segunda pasada, se genera el código máquina, las direcciones para la etiquetas se toman de la tabla de símbolos. Luego código y tabla se guardan en el archivo objeto.

```
gcc -c prog.s -o prog.o
```

```
gcc -O1 -g -c prog.c -o prog.o
```



Ensamblado

- Finalmente el ensamblador va a crea un formato de archivo denominado objeto (.o) que contiene el código máquina y una serie de tablas e información adicional
- Esto permite al Enlazador unir todas las piezas



Ensamblado

Partes que componen al .o

1) Cabecera del archivo objeto (Object file header)

El "índice" del archivo. Describe el tamaño y la ubicación de las demás piezas que componen el archivo. Contiene información como:

- El tipo de arquitectura para la que fue compilado (en nuestro caso, ARM).
- El tamaño total del archivo.



Ensamblado

Partes que componen al .o

1) Cabecera del archivo objeto (Object file header)

2) Segmento de texto (Text segment)

Se encuentra el código de máquina.

Contiene las instrucciones binarias que el procesador ejecutará (ej: los ADD, LDR o STR resultantes de tu main.c).



Ensamblado

Partes que componen al .o

- 1) **Cabecera del archivo objeto (Object file header)**
- 2) **Segmento de texto (Text segment)**
- 3) **Segmento de datos estáticos (Static data segment)**

Contiene los datos que se asignan a variables.

- **Datos inicializados:** Variables globales o estáticas que tienen un valor inicial (ej: `int sensor_id = 5;`).
- **Datos no inicializados (BSS):** Espacio reservado para variables globales que arrancan en cero.



Ensamblado

Partes que componen al .o

- 1) Cabecera del archivo objeto (Object file header)**
- 2) Segmento de texto (Text segment)**
- 3) Segmento de datos estáticos (Static data segment)**
- 4) Tabla de símbolos (Symbol table)**

Es una tabla interna del archivo. Contiene los nombres de las etiquetas (funciones y variables globales) definidas en este archivo que podrían ser utilizadas por otros.

Ejemplo: Una función `config_adc()`, su nombre y dirección relativa aparecen aquí para que el Linker pueda encontrarla.



Ensamblado

Partes que componen al .o

- 1) Cabecera del archivo objeto (Object file header)**
- 2) Segmento de texto (Text segment)**
- 3) Segmento de datos estáticos (Static data segment)**
- 4) Tabla de símbolos (Symbol table)**
- 5) Tabla de relocación (Relocation information)**

Son las direcciones sin resolver.

Lista todas las instrucciones que dependen de una dirección absoluta de memoria que el ensamblador aún no conoce.

Esto permite al enlazador reemplazarlas (a las direcciones) cuando sepa la ubicación.



Ensamblado

Partes que componen al .o

- 1) Cabecera del archivo objeto (Object file header)**
- 2) Segmento de texto (Text segment)**
- 3) Segmento de datos estáticos (Static data segment)**
- 4) Tabla de símbolos (Symbol table)**
- 5) Tabla de relocación (Relocation information)**
- 6) Información de depuración (Debugging information)**

Información extra para depurar

Contiene por ejemplo la relación entre las líneas de código C y las instrucciones de máquina.

Permite al GDB ubicar un breakpoint en una línea de C o ver el nombre de una variable en lugar de su dirección hexadecimal.

Ensamblado

```
00000000 <sum>:
    int sum(int a, int b) {
    return (a + b); }
0:  e0800001    ADD R0, R0, R1
4:  e12fff1e    BX LR
00000008 <main>:
    int f, g, y; // global variables
    int sum(int a, int b);
    int main(void) {
8:  e92d4008    PUSH {R3, LR}
    f = 2;
C:  e3a00002    MOV R0, #2
10: e59f301c    LDR R3, [PC, #28] ; 34 <main+0x2c>
14: e5830000    STR R0, [R3]
    g = 3;
18: e3a01003    MOV R1, #3
1c: e59f3014    LDR R3, [PC, #20] ; 38 <main+0x30>
20: e5831000    STR R1, [R3]
    y = sum(f, g);
24: ebfffffe    BL 0 <sum>
28: e59f300c    LDR R3, [PC, #12] ; 3c <main+0x34>
2c: e5830000    STR R0, [R3]
    return y;
    }
30: e8bd8008    POP {R3, PC}
34: 00000000    .word 0x00000000
38: 00000000    .word 0x00000000
3c: 00000000    .word 0x00000000
```

SYMBOL TABLE:

00000000	l	d	.text	00000000	.text
00000000	l	d	.data	00000000	.data
00000000	g	F	.text	00000008	sum
00000008	g	F	.text	00000038	main
00000004	0	*	COM*	00000004	f
00000004	0	*	COM*	00000004	g
00000004	0	*	COM*	00000004	y

objdump -t prog.o

objdump -S prog.o

Ensamblado

```
00000000 <sum>:
    int sum(int a, int b) {
    return (a + b); }
0:  e0800001    ADD R0, R0, R1
4:  e12fff1e    BX LR
00000008 <main>:
    int f, g, y; // global variables
    int sum(int a, int b);
    int main(void) {
8:  e92d4008    PUSH {R3, LR}
    f = 2;
C:  e3a00002    MOV R0, #2
10: e59f301c    LDR R3, [PC, #28] ; 34 <main+0x2c>
14: e5830000    STR R0, [R3]
    g = 3;
18: e3a01003    MOV R1, #3
1c: e59f3014    LDR R3, [PC, #20] ; 38 <main+0x30>
20: e5831000    STR R1, [R3]
    y = sum(f,g);
24: ebfffffe    BL 0 <sum>
28: e59f300c    LDR R3, [PC, #12] ; 3c <main+0x34>
2c: e5830000    STR R0, [R3]
    return y;
    }
30: e8bd8008    POP {R3, PC}
34: 00000000    .word 0x00000000
38: 00000000    .word 0x00000000
3c: 00000000    .word 0x00000000
```

SYMBOL TABLE:

00000000	l	d	.text	00000000	.text
00000000	l	d	.data	00000000	.data
00000000	g	F	.text	00000008	sum
00000008	g	F	.text	00000038	main
00000004	0	*	COM*	00000004	f
00000004	0	*	COM*	00000004	g
00000004	0	*	COM*	00000004	y

Función **sum**
comienza en 0 long de 8

Función **main**
comienza en 8 long de 56

Las variables globales
tienen asignado donde guardar
el puntero pero no la dirección
del valor



Enlazado (*linking*)

- Un proyecto en general están compuesto por varios archivos fuentes, librerías y archivos de inicialización.
 - Si se cambia uno de ellos es innecesario re-compilar todos.
-
- El trabajo del enlazador, es juntar todos los archivos objeto que componen un proyecto en un archivo en código máquina ejecutable y asignar las direcciones de variables globales.
 - El enlazador reubica los datos y las funciones de los archivos objetos para que no se superpongan, usa la información de sus tabla de símbolos para ajustar el código según la dirección asignada a las funciones y variables.



Enlazado (*linking*)

En enlazador consta de tres pasos

1 - Colocación de secciones y resolución de símbolos

Toma los archivos objeto (main.o, crt.o, etc) y los combina. En este paso:

Identifica dónde termina cada sección (como .text o .data).

Une todas las funciones y variables dispersas.

Su tarea es determinar qué etiquetas se refieren a qué bloques de código o datos en el programa completo.



Enlazado (*linking*)

En enlazador consta de tres pasos

1 - Colocación de secciones y resolución de símbolos

2 - Resolución de referencias externas

En los archivos objeto, muchas direcciones de memoria no están resueltas (el compilador no sabe donde estarán las funciones de otros archivos o de las librerías)

El enlazador busca las definiciones de esos símbolos y rellena esos huecos.



Enlazado (*linking*)

En enlazador consta de tres pasos

- 1 - Colocación de secciones y resolución de símbolos**
- 2 - Resolución de referencias externas**
- 3 - Asignación de direcciones físicas (Relocalización)**

El enlazador asigna direcciones de memoria absolutas a cada sección basándose en las restricciones del hardware.

Por ejemplo

- Determina que el código irá a la dirección de la FLASH (0x08000000).
- Determina que las variables irán a la RAM (0x20000000).

Calcula valores críticos como el fin de la memoria para definir el Stack Pointer inicial.



Enlazado (*linking*)

El enlazador genera un archivo ejecutable que puede ejecutarse en un ordenador.

Normalmente, es el mismo formato que un archivo objeto, pero con todas las referencias resueltas.

Ojo

Es posible tener archivos con referencias sin resolver, (rutinas de librerías dinámicas).

Estas librerías dinámicas, permiten reutilizar código y aprovechar memoria, pero requiere un SO que resuelva estas direcciones

Enlazado

```
00008390 <sum>:
    int sum(int a, int b) {
        return (a + b); }
8390: e0800001 ADD R0, R0, R1
8394: e12fff1e BX LR
00008398 <main>:
    int f, g, y; // global variables
    int sum(int a, int b);
    int main(void) {
8398: e92d4008 PUSH {R3, LR}
        f = 2;
839c: e3a00002 MOV R0, #2
83a0: e59f301c LDR R3, [PC, #28] ; 83c4 <main+0x2c>
83a4: e5830000 STR R0, [R3]
        g = 3;
83a8: e3a01003 MOV R1, #3
83ac: e59f3014 LDR R3, [PC, #20] ; 83c8 <main+0x30>
83b0: e5831000 STR R1, [R3]
        y = sum(f,g);
83b4: ebfffff5 BL 8390 <sum>
83b8: e59f300c LDR R3, [PC, #12] ; 83cc <main+0x34>
83bc: e5830000 STR R0, [R3]
        return y;
    }
83c0: e8bd8008 POP {R3, PC}
83c4: 00010570 .word 0x00010570
83c8: 00010574 .word 0x00010574
83cc: 00010578 .word 0x00010578
```

SYMBOL TABLE:

000082e4	l	d	.text	00000000	.text
00010564	l	d	.data	00000000	.data
00008390	g	F	.text	00000008	sum
00008398	g	F	.text	00000038	main
00010570	g	0	.bss	00000004	f
00010574	g	0	.bss	00000004	g
00010578	g	0	.bss	00000004	y

objdump -S -t prog

Enlazado

```
00008390 <sum>:
    int sum(int a, int b) {
        return (a + b); }
8390: e0800001 ADD R0, R0, R1
8394: e12fff1e BX LR
00008398 <main>:
    int f, g, y; // global variables
    int sum(int a, int b);
    int main(void) {
8398: e92d4008 PUSH {R3, LR}
        f = 2;
839c: e3a00002 MOV R0, #2
83a0: e59f301c LDR R3, [PC, #28] ; 83c4 <main+0x2c>
83a4: e5830000 STR R0, [R3]
        g = 3;
83a8: e3a01003 MOV R1, #3
83ac: e59f3014 LDR R3, [PC, #20] ; 83c8 <main+0x30>
83b0: e5831000 STR R1, [R3]
        y = sum(f,g);
83b4: ebfffff5 BL 8390 <sum>
83b8: e59f300c LDR R3, [PC, #12] ; 83cc <main+0x34>
83bc: e5830000 STR R0, [R3]
        return y;
    }
83c0: e8bd8008 POP {R3, PC}
83c4: 00010570 .word 0x00010570
83c8: 00010574 .word 0x00010574
83cc: 00010578 .word 0x00010578
```

SYMBOL TABLE:

000082e4	l	d	.text	00000000	.text
00010564	l	d	.data	00000000	.data
00008390	g	F	.text	00000008	sum
00008398	g	F	.text	00000038	main
00010570	g	0	.bss	00000004	f
00010574	g	0	.bss	00000004	g
00010578	g	0	.bss	00000004	y

Las funciones tiene
dirección final

Las variables globales
tienen asignado la dirección
del valor

Enlazado

00008390 <sum>:

```
int sum(int a, int b) {  
    return (a + b); }
```

8390: e0800001 ADD R0, R0, R1

8394: e12fff1e BX LR

00008398 <main>:

```
int f, g, y; // global variables  
int sum(int a, int b);  
int main(void) {
```

8398: e92d4008 PUSH {R3, LR}

```
    f = 2;
```

839c: e3a00002 MOV R0, #2

83a0: e59f301c LDR R3, [PC, #28] ; 83c4

83a4: e5830000 STR R0, [R3]

```
    g = 3;
```

83a8: e3a01003 MOV R1, #3

83ac: e59f3014 LDR R3, [PC, #20] ; 83c8

83b0: e5831000 STR R1, [R3]

```
    y = sum(f,g);
```

83b4: ebffffff5 BL 8390 <sum>

83b8: e59f300c LDR R3, [PC, #12] ; 83cc

83bc: e5830000 STR R0, [R3]

```
    return y;
```

```
}
```

83c0: e8bd8008 POP {R3, PC}

83c4: 00010570 .word 0x00010570

83c8: 00010574 .word 0x00010574

83cc: 00010578 .word 0x00010578

**Desde
binario**

00000000 <sum>:

```
int sum(int a, int b)  
return (a + b); }
```

0: e0800001 ADD R0, R0, R1

4: e12fff1e BX LR

00000008 <main>:

```
int f, g, y; // global variables  
int sum(int a, int b);  
int main(void) {
```

8: e92d4008 PUSH {R3, LR}

```
    f = 2;
```

C: e3a00002 MOV R0, #2

10: e59f301c LDR R3, [PC, #28] ; 34

14: e5830000 STR R0, [R3]

```
    g = 3;
```

18: e3a01003 MOV R1, #3

1c: e59f3014 LDR R3, [PC, #20] ; 38

20: e5831000 STR R1, [R3]

```
    y = sum(f,g);
```

24: ebffffff5 BL 0 <sum>

28: e59f300c LDR R3, [PC, #12] ; 3c

2c: e5830000 STR R0, [R3]

```
    return y;
```

```
}
```

30: e8bd8008 POP {R3, PC}

34: 00000000 .word 0x00000000

38: 00000000 .word 0x00000000

3c: 00000000 .word 0x00000000

**Desde
objeto**



Cargador

Es el encargado de colocar el archivo ejecutable en la memoria principal del computador para que comience la ejecución.

Pasos del Cargador (SO por ejemplo UNIX):

- Lee el encabezado del archivo ejecutable para determinar el tamaño de los segmentos de texto (código) y datos.
- Crea un espacio de direccionamiento lo suficientemente grande para el programa.
- Copia las instrucciones y los datos en la memoria.
- Inicializa los registros de la CPU (como el Stack Pointer).
- Salta a la rutina de inicio, la cual llama finalmente a la función principal del programa (como main).



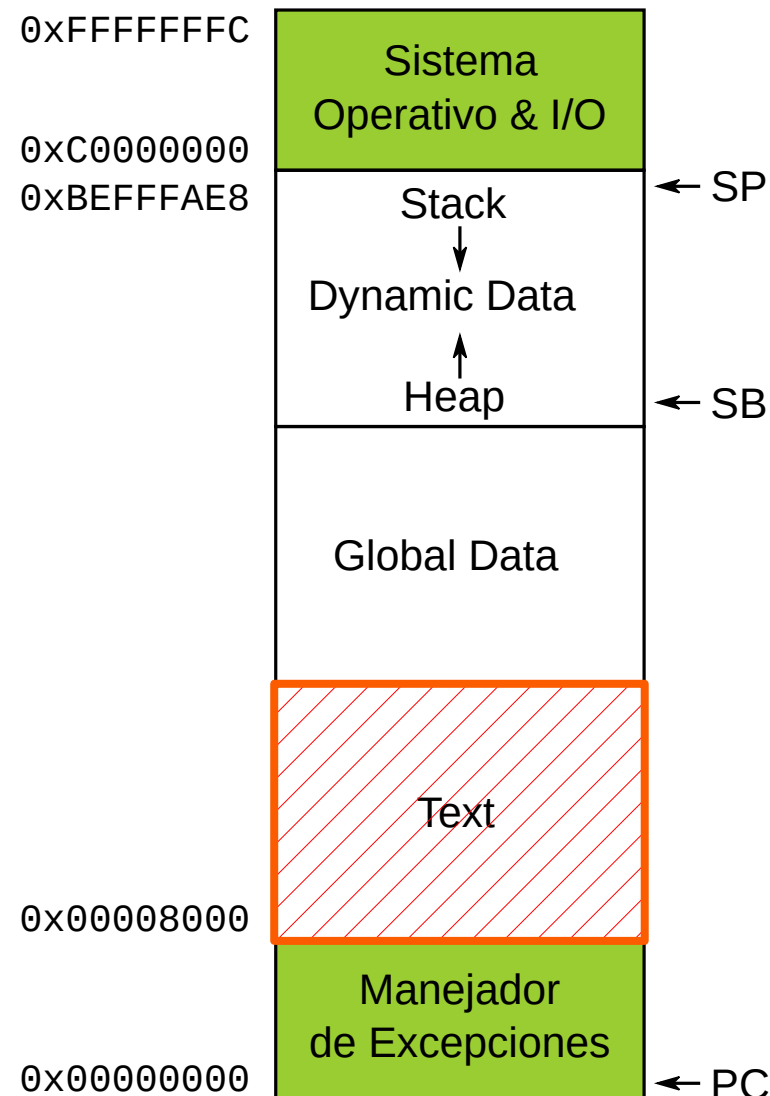
Mapa de Memoria

El programa ahora está finalmente en la memoria.

Para el caso de un ARM de 32 bits la misma tendrá la siguiente estructura

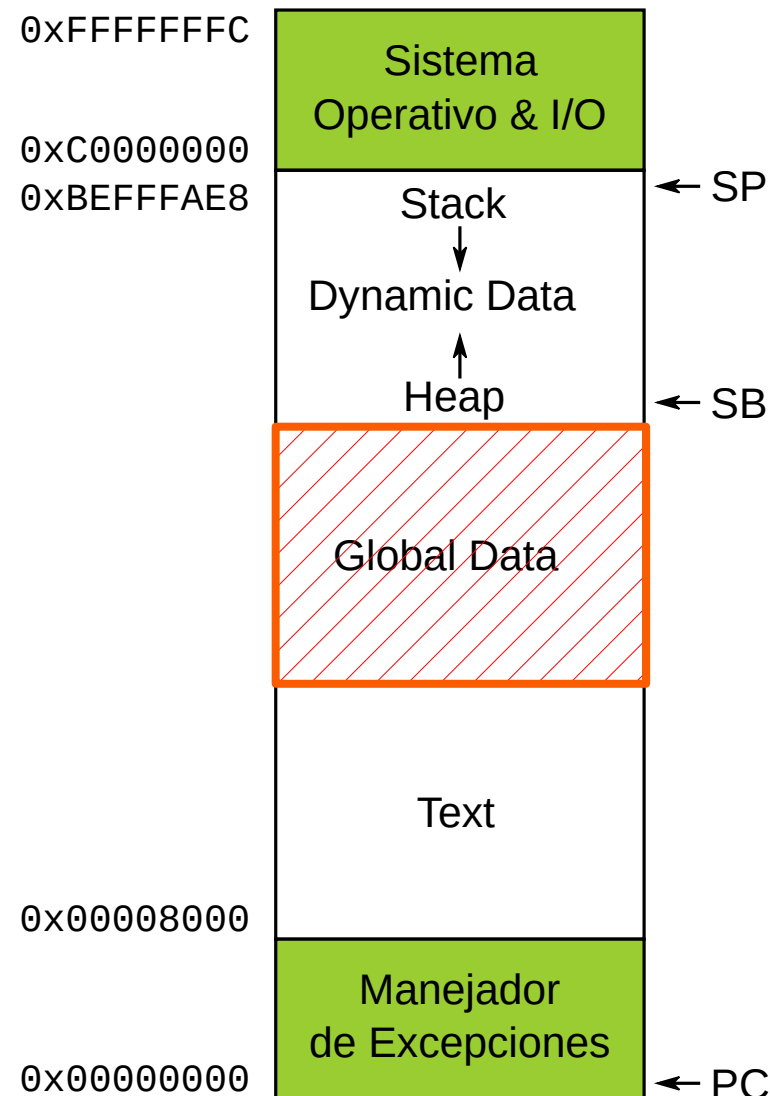
- El espacio de direcciones en ARM posee 32 bit.
- Esto permite 2^{32} bytes (4GB).
- Las direcciones de los word entonces serán múltiplo de 4 en posiciones de 0 a 0xFFFFFFF0.
- Finalmente el programa resuelto en 4 partes o segmentos.

Mapa de memoria



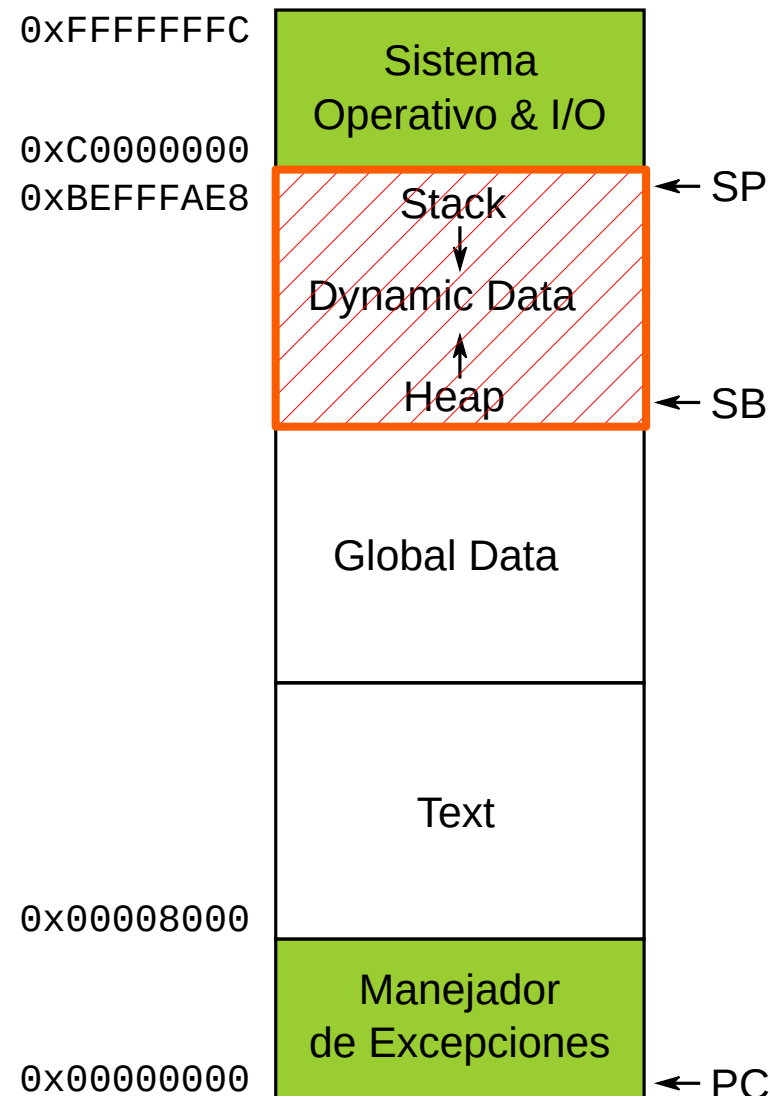
- **Segmento de texto.** segmento RO (*Read Only*), guarda el código máquina del programa, puede incluir literales (constantes) y datos de solo lectura.

Mapa de memoria



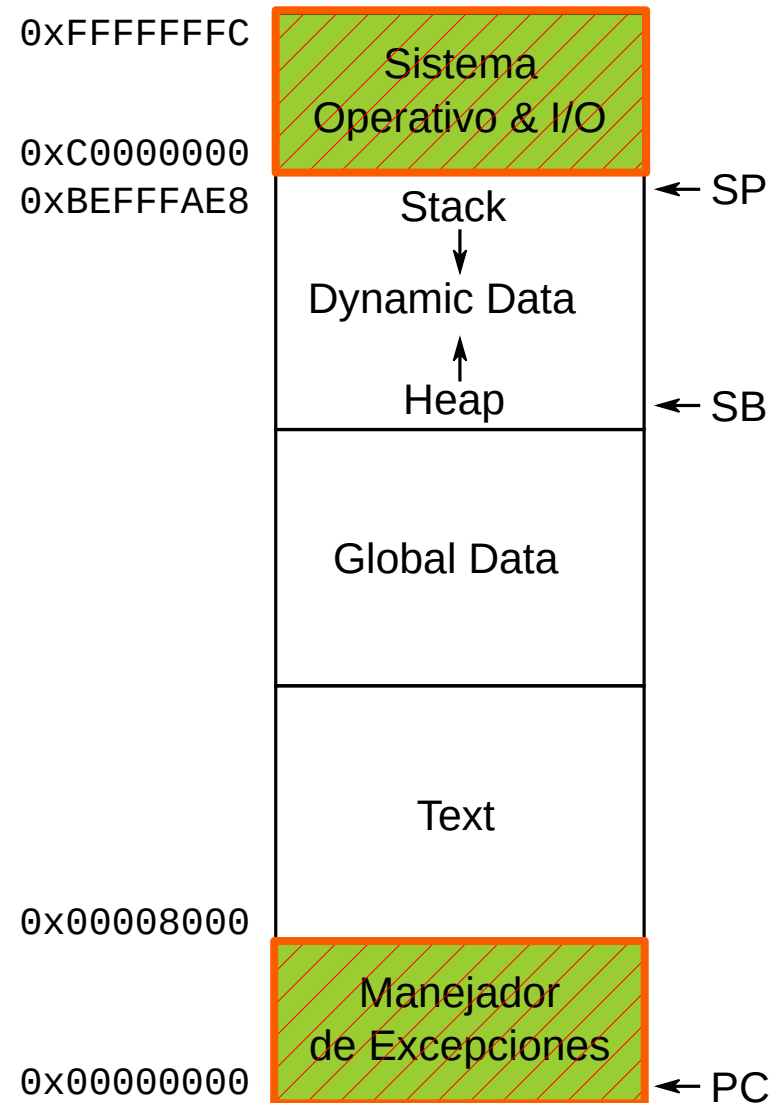
- **Segmento de texto.**
- **Segmento de datos globales.** segmento RW (*Read/Write*), guarda las variables globales, estas variables son ubicadas en la memoria antes de que el programa comience, ARM comúnmente usa el registro R9 (SB) como *Static Basic Pointer*, o puntero base para acceder a estas variables.

Mapa de memoria



- **Segmento de texto.**
- **Segmento de datos globales.**
- **Segmento de Datos Dinámicos.** Posee el Stack y el Heap este último encargado de almacenar las variables dinámicas generadas con `malloc ( C )` o `new ( C++ )`, generalmente crece en forma inversa al stack, el asignador de memoria es el encargado de no corromper la memoria, evitando el cruce con el Stack.

Mapa de memoria



- **Segmento de texto.**
- **Segmento de datos globales.**
- **Segmento de Datos Dinámicos.**
- **Manejador de Excepciones, OS y segmento de I/O.**
No es parte realmente del programa
La parte mas baja del mapa es reservada para el vector de excepciones y para los manejadores de excepciones, mientras que la parte mas alta para el sistema operativo y los dispositivos de I/O.



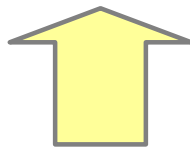
LinkerScript

El enlazado es la etapa final y crítica en la creación del ejecutable.

Es donde todas las piezas (archivos objeto y bibliotecas) se unen para formar el programa que correrá en nuestra computadora o microcontrolador.

Una de sus funciones principales del enlazado es asignar las direcciones físicas exactas donde residirán el código y las variables.

- Esto no es problema en una PC donde el Sistema Operativo decide esto en tiempo de ejecución (Cargador).
- En un microcontrolador nosotros somos los responsables de definir ese mapa.



Aquí es donde aparece el Linker Script (.ld):



LinkerScript

Es el archivo de configuración para el Enlazador: Sin él, el enlazador desconoce las características de nuestro chip.

Alguna de sus funciones:

- **Límites Físicos:** Define dónde arranca y dónde termina la memoria (FLASH y RAM).
- **Orden de Secciones:** Le ordena al Linker por ejemplo ubicar la tabla de vectores al principio de todo.
- **Creación de Símbolos:** Define etiquetas como `_sdata` o `_esstack`, que luego nuestro código (o el Startup) usará como direcciones reales.



El Mapa de Memoria de la Blue Pill

Ejemplo:

El Cortex-M3 tiene un espacio de direccionamiento de 4GB, pero la Blue Pill solo usa una fracción:

- Flash: Inicia en 0x08000000 (64 KB).
- RAM: Inicia en 0x20000000 (20 KB).

El Linker Script debe indicar estos límites para el enlazador.



Estructura Básica: El comando MEMORY

Define los bloques físicos de memoria disponibles.

Sintaxis:

Código:

```
MEMORY {  
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K  
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 20K  
}
```

rx: Read/Execute (Flash). rwx: Read/Write/Execute (RAM).



El comando SECTIONS

Es el bloque donde le decimos al linker qué sección de los archivos objeto (.o) va a qué memoria física.

Sintaxis inicial:

Código:

```
SECTIONS {  
    .text : { *(.text) } > FLASH  
}
```

El asterisco * significa "*tomar todas las secciones .text de todos los archivos .o*".

Luego > **FLASH** estamos indicando el destino de ese bloque



La importancia del Orden

El hardware ARM busca la **Tabla de Vectores** al inicio de la Flash.

Por eso, el Linker Script debe forzar que esa sección sea la primera:

Código:

```
SECTIONS {  
    .text : {  
        KEEP(*(.isr_vector)) /* Tabla de vectores primero */  
        *(.text)             /* El resto del código */  
    } > FLASH  
    . . .
```

KEEP(*(.isr_vector)) no solo obliga a tomar la sección `.isr_vector` y colocarla primero, además KEEP asegura mantenerla a pesar de que no se llame desde ningún lado.



Relación con el Startup (crt.s)

El Startup es un archivo fundamental en aplicaciones que no corren sobre un SO, es el encargado de inicializar el hardware del microcontrolador, además mantiene una relación importante con el linkerscript en los siguientes puntos

- El archivo de startup define etiquetas como **.isr_vector**. El Linker Script buscara esa etiqueta y la ubica en la 1era posición **KEEP(*(.isr_vector))**
- Inicializa memoria y copia variables preinicializadas utilizando etiquetas asignadas en el Linker Script

Es la primera conexión entre el código ensamblador y la memoria física.



Ubicando el Stack Pointer (_esstack)

El microcontrolador necesita saber dónde empieza su pila (Stack).

Generalmente se ubica al final de la RAM.

Para el caso de BluePill, asignar el inicio de la pila se realiza en 2 pasos

1. Cálculo en el Script:

```
_esstack = ORIGIN(RAM) + LENGTH(RAM);
```

2. La posición de arranque del Stack se establece almacenando en la primera dirección de la memoria Flash (0x08000000) el valor de inicio de la pila.

En el Startup, se asigna _esstack en la 1era entrada de la tabla de vectores (.isr_vector)



El problema con la Sección .data

Las variables globales inicializadas (ej. `int x = 5;`) tienen dos problema:

- Deben guardarse en Flash (para no perderse el valor inicial al apagar).
- Deben ejecutarse en RAM (para poder cambiar su valor).



LMA vs. VMA

Para esto, se define dos tipos de memoria

- **LMA (Load Memory Address):** Dirección donde se guarda el dato.
- **VMA (Virtual Memory Address):** Dirección donde vive el dato mientras corre.
Esta es la dirección donde se asignarán los punteros.

El Linker Script gestiona esta dualidad usando la palabra clave “AT >” para LMA.

Ejemplo de Sección .data

Código

```
_sidata = LOADADDR(.data);  
.data : {  
    _sdata = .;           /* Inicio en RAM */  
    *(.data)  
    _edata = .;           /* Fin en RAM */  
} > RAM AT > FLASH
```

- . (punto) es el "Location Counter" actual del Linker.
- **_sidata = LOADADDR(.data);** Lee el valor de la dirección LMA donde se guardó .data
- **> RAM AT > FLASH** asigna espacio en RAM y guarda datos en FLASH

Es el Startup el que usa los símbolos _sdata, _edata y _sidata para mover los valores de Flash a RAM durante el arranque.



La Sección .bss

Aquí van las variables que valen cero al arrancar (ej. `int buffer[100];`).

No ocupan espacio en Flash (solo se reserva el espacio en RAM).

Código

```
.bss : {  
    _sbss = .;  
    *(.bss)  
    _ebss = .;  
} > RAM
```

Al igual que con `.data`, el Startup debe leer los símbolos `_sbss` y `_ebss` para poner a cero esa región de la RAM antes de llamar a `main()`



El Alignment (Alineación)

El Cortex-M3 accede más eficientemente a direcciones alineadas a 4 bytes (32 bits).

Usamos `ALIGN(4)` en el script para evitar errores de acceso a memoria.

```
. = ALIGN(4);
```



El Startup

Archivo fundamental en un entorno como un microcontrolador, donde no tenemos un SO que lea y ejecute una aplicación.

Llamado comunmente **crt.s** o **startup.s** si está escrito ensamblador o **startup.c** si está escrito en C, es el primer código que se ejecuta luego de un reset del microcontrolador.

Se encarga de prepara el hardware para que el lenguaje C pueda funcionar.

Sus tareas serán:



El Startup

1) Establecer el Punto de Entrada (Reset Vector)

Le indica al hardware el código a ejecutar al arrancar el procesador

2) Inicializar el Stack Pointer (SP)

Vimos que el LinkerScript calcula el SP, es el Startup el que finalmente lo carga.

3) Inicializar el Segmento de Datos (.data)

Las etiquetas creadas en el LinkerScript finalmente son utilizadas aquí para copiar mediante bucles los valores de inicio de la flash a la RAM.

4) Limpiar el Segmento BSS (.bss)

Por estándar de ANSI C, todas las variables globales no inicializadas deben valer cero al arrancar.

El Startup se encarga de borrar cada una de estas variables.



El Startup

5) Configuración Básica de Hardware (Opcional)

Generalmente agrupada en la función como SystemInit() donde se configura el PLL y que el microcontrolador corra a la velocidad máxima (72 MHz en la Blue Pill) en lugar de la velocidad de arranque.

6) El Salto Final al main()

Una vez que la RAM está lista y la pila configurada, el Startup ejecuta un salto (generalmente un **bl main** en ensamblador) para entregarle el control total a la aplicación.



El Startup mas sencillo

```
.syntax unified
.cpu cortex-m3
.thumb
.global _esstack

.word _esstack ; puntero de stack
.word _reset   ; puntero de la función de reset
.thumb_func
_reset:
    bl main
    b .
```

Un Startup mas completo

```
/* crt.s - Startup para STM32F103 (Blue Pill) */  
.syntax unified  
.cpu cortex-m3  
.thumb  
  
/* Importamos los símbolos definidos en el Linker Script (.ld) */  
.word _sidata /* Inicio de datos en Flash (LMA) */  
.word _sdata /* Inicio de datos en RAM (VMA) */  
.word _edata /* Fin de datos en RAM */  
.word _sbss /* Inicio de BSS en RAM */  
.word _ebss /* Fin de BSS en RAM */  
.word _esstack /* Fin de la RAM (Inicio del Stack) */  
  
.section .isr_vector, "a", %progbits /* sección de interrupciones */  
.type g_pfnVectors, %object
```

Un Startup mas completo

```
/* Definición de la Tabla de Vectores */
```

```
g_pfnVectors:
```

```
    .word _esstack          /* 0x00: Puntero de Pila inicial */  
    .word Reset_Handler    /* 0x04: Punto de entrada al Reset */  
    .word NMI_Handler  
    .word HardFault_Handler  
    /* ... el resto de las interrupciones ... */
```

```
.section .text.Reset_Handler  
.weak Reset_Handler  
.type Reset_Handler, %function
```

```
Reset_Handler:
```

```
    /* Aqui se copian la sección .data de la FLASH a RAM */  
    /* La info son tomados del linkerscript */
```

```
    ldr r0, =_sdata         /*comienzo en RAM del .data */  
    ldr r1, =_edata         /*fin en RAM del .data */  
    ldr r2, =_sdata         /*comienzo en FLASH del .data */  
    movs r3, #0  
    b LoopCopyDataInit
```



Un Startup mas completo

CopyDataInit:

```
ldr r4, [r2, r3]
str r4, [r0, r3]
adds r3, r3, #4
```

LoopCopyDataInit:

```
adds r4, r0, r3
cmp r4, r1
blo CopyDataInit
```

/* Limpiar la sección .bss (llenar con ceros) */

```
ldr r2, =_sbss
ldr r4, =_ebss
movs r3, #0
b LoopFillZeroBss
```



Un Startup mas completo

```
FillZerobss:
    str r3, [r2]
    adds r2, r2, #4

LoopFillZerobss:
    cmp r2, r4
    blo FillZerobss

/* Salto a la función principal en C */
    bl main
    bx lr /* Por seguridad, si main retorna */

.size Reset_Handler, . - Reset_Handler
```



¿ Preguntas ?